

Gazebo / MuJoCo

▼ Gazebo (Classic)

실제 장비 없이 가상 환경에서 테스트하는 시뮬레이터

실제 Gazebo로 하는 일은 아래와 같다.

- 실제 로봇 투입 전 검증
- 센서 데이터 테스트
- 자율주행 알고리즘 검증
- 로봇팔 동작 시뮬레이션

▼ MuJoCo

MuJoCo는 로봇틱스, 애니메이션등의 물리 시뮬레이션을 위해 사용하는 오픈소스 물리 엔진.

쉽게 말하면, 로봇이 실제 세상에서 움직이는 것처럼 “물리 법칙을 계산해 주는 엔진”이다.

MuJoCo Python 바인딩은 Google DeepMind가 관리하는 공식 Python 패키지이며,

```
pip install mujoco
```

방식으로 설치할 수 있고 Python 3.10 이상을 전제로 함.

MuJoCo는 다음과 같은 것들을 계산한다.

관절 운동	1번 관절을 30도 돌리면 팔 끝이 어디로 가는가
중력	팔이 무거우면 아래로 처지는가
충돌	물체와 물체 또는 로봇 손이 부딪혔는가
마찰	물체를 잡았을 때 미끄러지는가
토크	모터가 어느 정도 힘을 내야 하는가
센서값	관절 위치, 속도, 힘, 접촉 정보

🌳 미리보는 최종 파일 tree

우리는 Docker를 사용해서 **MuJoCo + Python + Menagerie** 모델을 실행할 것.

```
Windows 10
├─ Docker Desktop
│   └─ Ubuntu 기반 Python 컨테이너
│       ├── Python
│       ├── mujoco==3.6.0
│       ├── mujoco_menagerie
│       ├── Franka Panda / UR5e / KUKA 등 로봇팔 MJCF 모델
│       └─ Python 시뮬레이션 코드
```

📁 사전 파일 작업

👉 이런 식으로 미리 폴더/파일들을 만들어야 함.

아래 코드들 참고 (필요 기능 추가 및 수정 할 것)

- app
- data
- docker
- docker-compose.yml
- Dockerfile
- requirements.txt

▼ Dockerfile / docker-compose.yml / requirements.txt 코드

Dockerfile

```
# Dockerfile

# Python 3.11 기반 이미지 사용
# MuJoCo 공식 Python 패키지는 Python 3.10 이상을 전제로 하므로,
# 교육 환경에서는 Python 3.11을 안정적인 기준으로 사용한다.
FROM python:3.11-slim

# 컨테이너 내부 작업 폴더
WORKDIR /workspace

# MuJoCo 실행과 렌더링에 필요한 Linux 패키지 설치
# libgl1, libegl1, libosmesa6 등은 MuJoCo 렌더링/오프스크린 렌더링 오류를 줄이기 위해 설치한다.
RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    vim \
    nano \
    tree \
    bash \
    terminator \
    build-essential \
    libgl1 \
    libegl1 \
    libgles2 \
    libosmesa6 \
    libglfw3 \
    libgl2.0-0 \
    libxrender1 \
    libxext6 \
    libsm6 \
    ffmpeg \
    && rm -rf /var/lib/apt/lists/*

# pip 최신화
RUN python -m pip install --upgrade pip setuptools wheel

# Python 패키지 목록 복사 후 설치
COPY requirements.txt /workspace/requirements.txt
RUN pip install --no-cache-dir -r /workspace/requirements.txt

# 기본 실행 셸
CMD ["bash"]
```

docker-compose.yml

```
# docker-compose.yml

services:
  mujoco:
    build:
      context: .
      dockerfile: Dockerfile

    container_name: mujoco_arm_lab

    working_dir: /workspace

    volumes:
      # Windows 프로젝트 폴더 전체를 컨테이너 /workspace에 연결
      - ../workspace

    environment:
      # 화면 없이 렌더링할 때 안정적인 방식
      # 나중에 이미지/영상 저장 실습에서 사용
      - MUJOCO_GL=osmesa

      # Python 로그를 즉시 출력
      - PYTHONUNBUFFERED=1

    stdin_open: true
    tty: true
```

requirements.txt

```
# requirements.txt

# MuJoCo 및 시뮬레이션 핵심
mujoco>=3.1.0      # 최신 기능을 위해 최소 버전 이상 권장
numpy<2.0.0        # MuJoCo/OpenCV와의 호환성을 위해 1.x대 권장

# 시각화 및 영상 처리
matplotlib          # 데이터 그래프 시각화
opencv-python       # 실시간 영상 처리 및 렌더링 결과 제어

# 영상 저장 (코덱 포함)
imageio              # 이미지/영상 입출력
imageio[ffmpeg]      # 고품질 mp4 저장을 위한 필수 코덱
imageio[pyav]        # 고성능 영상 처리를 위한 백엔드
```

이후는 `docker-compose build / up-d / exec` 해서 들어가주면 됨. (모르면 Docker 노션 참고)

MuJoCo Menagerie 저장소 다운로드

```
cd /workspace
git clone https://github.com/google-deeppmind/mujoco_menagerie.git
```

https://github.com/google-deeppmind/mujoco_menagerie

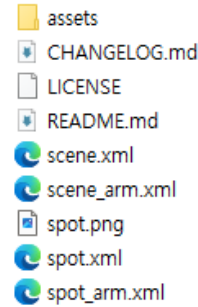
(참고) 위 깃허브를 다운받는 거임.

이후, mujoco_menagerie 라는 이름의 폴더가 생기고 그 안에 수많은 로봇 폴더들이 있을거임.

mujoco_menagerie/assets



mujoco_menagerie/spot



그 안에 여러 파일들 중에 " [로봇이름].xml / scene.xml " 파일이 있는 걸 알 수 있음.

[로봇이름].xml

[로봇이름].xml 은 로봇 자체의 정의에 가깝다.

쉽게 얘기하면 " 로봇팔 본체 설계도 ".

body, joint, geom, assets 등 존재.

구성	의미
body	로봇 링크 구조
joint	관절
geom	충돌/시각 형상
actuator	관절을 움직이는 모터/제어기
asset	mesh, texture 등
default	반복 설정 기본값

모델에 센서, 장비등을 수정하려면 [로봇이름].xml,

레이어나 조명, 계단, 주변 물체를 수정하려면 scene.xml을 고쳐야 함.

scene.xml

scene.xml 은 로봇을 놓고 실행할 장면이다.

쉽게 얘기하면 " 로봇팔이 놓인 실습 공간 ".

include, worldbody, visual 등 존재.

구성	의미
include	panda.xml 로봇 모델 불러오기
worldbody	바닥, 조명, 카메라, 물체
visual	화면 표시 설정
statistic	카메라 중심, 모델 범위 등
keyframe	초기 자세 또는 예제 자세

✓ MUJOCO_GL = osmesa 확인

MuJoCo 렌더링 백엔드 설정을 확인하기 위해 터미널 창에 아래 명령어로 확인해보기.

```
echo $MUJOCO_GL  
  
# 출력 : osmesa
```

OSMesa : 화면이 없는 환경에서도 '메모리 상에서' 그래픽을 그려주는 도구
